Systems Programming

Achieve maximum effect with minimum means.

What is a “systems” programming language?

Needs to be “low level.” What does this mean?

- Simply that the language deals with many of the same things that computers do, namely, characters, numbers, addresses and memory.

Used to create “systems”

- Like operating systems!
- Like compilers!
- Many other applications which require strict, explicit memory management

Survey of C

The C Programming Language

C is a *low level* **imperative** programming language initially designed and implemented by Dennis Ritchie for the UNIX operating system.

- Note, it is traditional for the linux OS to use C, although rust is also accepted today.

C is very versatile, doesn't provide many safety mechanisms. Common errors?

segfault!

Brian Kernighan and **Dennis Ritchie** are the authors of perhaps the most famous programming language book, *The C Programming Language.*

Features

Strongly typed

Control flow that you are familiar with (because java used it 20 years later)

Access to low level data; pointers, memory addresses etc.

No storage allocation facility other than “stack” management of local variables (malloc is a library function!)

No heap garbage collection.

(what about non-heap objects?)

Examples

Slide escape, lets program!

- Simple things
- Pointers
- Pass by

On pointers…

Pointers have been lumped with the `goto` statement as a marvelous way to create impossible-to-understand programs.

- Dennis Ritchie

Go

Modern Systems Programming

The Go Programming Language

- Go was designed by Robert Griesemer, Rob Pike, Ken Thompson at Google in 2009.
  - Rob and Ken worked on the UNIX operating system at Bell labs in the 1970's.
  - Griesemer developed Pascal and also implemented the V8 javascript interpreter at Google.

- Go is a imperative, block structured, statically scoped, strongly typed language.

Achieve maximum effect with minimum means.

More features

Compiles to machine code

Has **garbage collection**!

Provides a mechanism (implicit though) for allocating memory on the heap

Arrays and maps are built in types (C just got syntax for Arrays)

Higher order functions with closures and early binding (you can program functionally too!) More on those later this semester

Built in concurrency (with the `go` statement) and communication (channels)

Strange things

- Function definitions can define the return variable
- Multiple return values
- Types come after variable names, not before
- Only has for loops
- Lowercase type names imply package privacy
- Type inference (`:=` vs `=`)
- Semicolons?
- Interfaces and embedded types
- That annoying thing that happens when you save

Object Orientation

Go is object oriented, however, not at all like you are used to.

Go programs can use a style of Object Orientation called ‘composition.’

Interfaces describe functionality, not identity.

Single method interfaces should be named with by the method + `‘er’`

`fmt.Stringer` for example.

There are no “classes” or other explicit constructs which indicate OOP.

It does have `package`, `interface` and `struct` though…

Shape example: /andre/examples/go/oo-example

Testing

Go has a built in test framework, it's really handy! (no libraries like JUnit or anything required).

To use it, name functions that are to be the test functions as “TestXyz(*testing.T) inside a file called xyz_test.go and the command go test will execute the tests!

More information here: https://pkg.go.dev/testing

“Idiomatic” Go

Idiomatic means “...like natural to a native speaker”

So, how do we program like natural Golang programmers?

How to write go code

Effective Go

Examples

Finally, lets program! (all found here)

- We did:
  - Sleepsort
  - The shape OO example

Thoughts on learning Go

'algorithm' for learning the language:

1. Make sure that you can run the class examples! One example was provided by Dr. Buffenbarger (sleep sort). Its found here.
2. The Go lang website is good. Here is a getting started tutorial.
    1. You might run into "A Tour of Go". That is good, but it's not very directed and I find not thorough enough to help much.
3. When you come to working on the project, how do you know you are working idiomatically? Cross-reference the features you are writing with the official Go recommendation. <- HIGHLY RECOMMENDED for doing things 'right' as a go programmer.
4. Of course, if all else fails, you can consult the manual.

I will recommend that you don't search the internet very much with this project, or with other languages. The reason is not that I don't want you to find resources, but that it can lead to confusing, conflicting recommendations. The problem we have is simple. Keep it that way.

Concurrency

Go threads are called goroutines

Technically, these are examples of virtual threads, and many more can be created than the Operating System would typically be able to manage efficiently.

Channels provide statically typed, atomic, and unbuffered or buffered communication between goroutines.

Concurrency II

In computer science, a thread (of execution) is a component of a process which can be managed independently with a scheduler.

Typically, the scheduler is part of the operating system. In Go, the thread scheduler is part of the program runtime.

Because threads share memory, they can be used to parallelize tasks. This can increase a programs efficiency.

Example: *Parallel Merge Sort*

Concurrency III

There is a problem with sharing memory.

An action in a high level language does not typically take a single CPU instruction.

```
b = 1 + b
```

```
load b
```

```
add b 1
```

```
store b
```

If one thread load b's value before the other can store the increased value…

This is called a race condition

An action taken by the computer which is indivisible into separate steps is atomic

Concurrency IV

There are many inventions which can prevent race conditions:

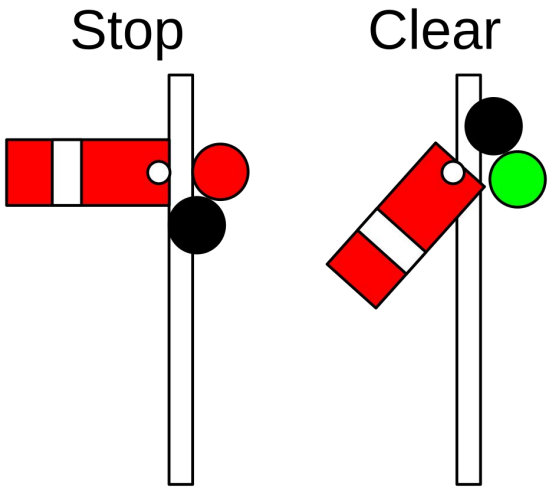
Mutex (for mutual exclusion)

Semaphore

Monitor

Go thinks about the problem differently.

Communicate by sharing messages, not data.



By Traced by Stannered - Own work, CC BY-SA 3.0, https://commons.wikimedia.org/w/index.php?curid=1863505

Concurrency V

Channels in go provide message passing.

Channels are statically typed, and provide atomic reads or writes

They also come with a bit of fun syntax

The arrow operator will read from or send to a channel depending on its location in an expression.

```
var c chan int

c <- 1

b := <-c
```

Concurrency VI

Examples:

examples/go/channels

buff/classes/354/etc/sleepsort